WINDOW FUNCTIONS

1.Find Second highest salary:

```
SELECT *
FROM (
        SELECT * ,
        DENSE_RANK()OVER(ORDER BY SALARY DESC) AS RANK
        FROM EMPLOYEES
        ) AS E
WHERE E.RANK = 2;
```

- SELECT * (outer) → return all columns from the derived table for rows that match the outer WHERE.
- FROM (...) AS E → derived (inline) view; we compute ranks inside and then filter outside.
- SELECT *, (inner) → return every column from EMPLOYEES to the derived result.
- DENSE_RANK() OVER (ORDER BY SALARY DESC) AS RANK → compute a rank based on salary, highest salary → rank 1, next distinct salary → rank 2. DENSE_RANK does not skip ranks on ties (if two people share top salary both get rank=1; the next distinct salary is rank=2).
- WHERE E.RANK = 2 → keep only rows whose salary is the second distinct highest. (If fewer than 2 distinct salaries → no rows.)

2.Find Second highest salary FOR EACH Department:

```
SELECT  *
FROM (
        SELECT  * ,
        DENSE_RANK()OVER(PARTITION BY Department_Id ORDER BY SALARY DESC) AS RANK
        FROM EMPLOYEES
        ) AS E
WHERE E.RANK = 2;
```

Same structure as (1) but:

- PARTITION BY Department_Id → the ranking restarts for each department (i.e., rank = 1 is highest salary inside each department).
- ORDER BY SALARY DESC → highest first inside each partition.
- WHERE E.RANK = 2 → returns employees who have the second distinct highest salary in their department. Ties at the top are handled by DENSE_RANK (all tied top earners get rank=1; the next distinct salary gets rank=2).

3.Find the oldest joinee department wise using LAG Analytic function:

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
```

```
  Hire_date,
  LAG(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date) AS PREV_HIREDATE
FROM EMPLOYEES;
```

- SELECT Employee_Id, First_Name, Department_Id, Hire_date → the columns we want to see.
- LAG(Hire_date) → returns the previous row's Hire_date according to the ordering inside the window.
- OVER (PARTITION BY Department_Id ORDER BY Hire_date) → the window is limited to each Department_Id, and rows are ordered ascending by Hire_date (earliest → latest).
- AS PREV_HIREDATE → alias for the previous-hire-date value.
- Behavior / how to get oldest: because ordering is ascending, the first row per partition is the oldest hire — for that row LAG(...) returns NULL. So to select oldest joinee only, add WHERE PREV_HIREDATE IS NULL around/after this.

4.Find the newest joinee department wise using LAG Analytic function:

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
  Hire_date,
  LAG(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date DESC) AS NEXT_HIREDATE
FROM EMPLOYEES
ORDER BY Department_Id, Hire_date;
```

--OR--

```
SELECT *
FROM(
   SELECT
     Employee_Id,
     First_Name,
     Department_Id,
     Hire_date,
     LAG(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date DESC) AS NEXT_HIREDATE
   FROM EMPLOYEES )
WHERE NEXT_HIREDATE IS NULL;
```

- LAG(Hire_date) ... ORDER BY Hire_date DESC → the rows are sorted newest → oldest inside each department.
- LAG(...) returns the previous row in that descending order, so for the newest row (first in DESC order) LAG will be NULL.
- AS NEXT_HIREDATE → name is a bit semantic — because we ordered DESC, the LAG value is actually the hire date of the more recent row relative to the current row. For the very newest, it's NULL.
- ORDER BY Department_Id, Hire_date (final output sorting) → just controls final result display (ascending by Hire_date), not the window ordering.

- Alternate (filter): wrapping the analytic in a subquery and filtering WHERE NEXT_HIREDATE IS NULL returns only the newest per dept.

5.Find the oldest joinee department wise using LEAD Analytic function:

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
  Hire_date,
  LEAD(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date DESC) AS PREV_HIREDATE
FROM EMPLOYEES
ORDER BY Department_Id, Hire_date;
```

--OR--

```
SELECT   *
FROM(
   SELECT
     Employee_Id,
     First_Name,
     Department_Id,
     Hire_date,
     LEAD(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date DESC) AS PREV_HIREDATE
   FROM EMPLOYEES)
WHERE PREV_HIREDATE IS NULL;
```

- LEAD(Hire_date) → looks ahead (next row) according to the specified ordering.
- ORDER BY Hire_date DESC → rows are newest → oldest. For the oldest row (last in DESC order) there is no next row, so LEAD returns NULL.
- AS PREV_HIREDATE → naming again semantic; in this setup PREV_HIREDATE IS NULL identifies the oldest joinee.
- Use the subquery + WHERE PREV_HIREDATE IS NULL to select only the oldest per department.

6..Find the newest joinee department wise using LEAD Analytic function:

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
  Hire_date,
  LEAD(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date) AS NEXT_HIREDATE
FROM EMPLOYEES;
```

--OR--

```
SELECT *
FROM(
   SELECT
     Employee_Id,
     First_Name,
     Department_Id,
     Hire_date,
     LEAD(Hire_date) OVER(PARTITION BY Department_Id ORDER BY Hire_date) AS NEXT_HIREDATE
   FROM EMPLOYEES)
WHERE NEXT_HIREDATE IS NULL;
```

- ORDER BY Hire_date → ascending (oldest → newest).
- LEAD(Hire_date) returns the next row's hire date: for the newest row there is no next row so LEAD(...) = NULL.
- WHERE NEXT_HIREDATE IS NULL (when used) → identifies the newest joinee per department.

7.RANK and DENSE_RANK:

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
  Salary,
  RANK() OVER(PARTITION BY Department_Id ORDER BY Salary) AS SAL_RANK
FROM EMPLOYEES;
```

--OR--

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
  Salary,
  DENSE_RANK() OVER(PARTITION BY Department_Id ORDER BY Salary) AS SAL_RANK
FROM EMPLOYEES;
```

- PARTITION BY Department_Id → ranking restarts per department.
- ORDER BY Salary → lowest salary → rank 1.
- RANK() → assigns equal ranks to ties but skips rank numbers after ties (e.g., salaries 100,100,90 → ranks 1,1,3).
- DENSE_RANK() (alternative) → assigns equal ranks to ties but does not skip numbers (same example → ranks 1,1,2).
- Choose RANK or DENSE_RANK based on whether you want gaps after ties.

8.Find employee with MAX salary department wise:

```
SELECT *
FROM(
   SELECT
     Employee_Id,
     First_Name,
     Department_Id,
     Salary,
     RANK() OVER(PARTITION BY Department_Id ORDER BY Salary DESC) AS SAL_RANK
   FROM EMPLOYEES
)
WHERE SAL_RANK =1;
```

- ORDER BY Salary DESC inside RANK() → highest salary → rank 1.
- WHERE SAL_RANK = 1 → returns top earner(s) per department. If multiple employees tie for max salary, all return.

9.Find employee with MIN salary department wise:

```
SELECT *
FROM(
   SELECT
     Employee_Id,
     First_Name,
     Department_Id,
     Salary,
     RANK() OVER(PARTITION BY Department_Id ORDER BY Salary) AS SAL_RANK
   FROM EMPLOYEES
)
WHERE SAL_RANK =1;
```

- ORDER BY Salary (ascending) inside RANK() → lowest salary → rank 1.
- WHERE SAL_RANK = 1 → returns the lowest-paid employee(s) per department (ties included).

10.Find the difference between the salary of an employee and max salary of the employee in the department:

```
SELECT
  Employee_Id,
  First_Name,
  Department_Id,
  Salary,
  MAX(Salary) OVER(PARTITION BY Department_Id ) AS MAX_SAL,
  (MAX(Salary) OVER(PARTITION BY Department_Id )-Salary ) AS SAL_DIFF
```

```
FROM EMPLOYEES
ORDER BY Employee_Id;
```

- MAX(Salary) OVER (PARTITION BY Department_Id) → a window function that computes the department's max salary and repeats it on every row of that department.
- AS MAX_SAL → alias for that repeated maximum.
- (MAX(...) - Salary) AS SAL_DIFF → numeric difference: how much less this employee earns than the department max (zero means this employee is one of the top earners).
- ORDER BY Employee_Id → sorts final output by Employee_Id (presentation only).

11.Find employee with MAX salary department wise without using RANK or DENSE_RANK:

```
SELECT *
FROM(
   SELECT
     Employee_Id,
     First_Name,
     Department_Id,
     Salary,
     MAX(Salary) OVER(PARTITION BY Department_Id ) AS MAX_SAL,
     (MAX(Salary) OVER(PARTITION BY Department_Id )-Salary ) AS SAL_DIFF
   FROM EMPLOYEES
)
WHERE SAL_DIFF = 0;
```

- Inner query computes MAX_SAL (department max) and SAL_DIFF.
- WHERE SAL_DIFF = 0 → selects rows where the employee's Salary equals the department max (so they are a max earner).
- This method returns all employees who tie for the max (no ranking needed). If you want a deterministic single row even when ties exist, use ROW_NUMBER() with a tie-breaker.